MATERIAŁ REFERENCYJNY

Databricks Fundamental

Kompendium do jednodniowego szkolenia

Altkom Akademia
Wydanie: październik 2026

Spis treści

MODUŁ 01	Platforma i workspace	3
MODUŁ 02	Architektura Spark	6
MODUŁ 03	ELT i ingestia	7
MODUŁ 04	Delta Lake	10
MODUŁ 05	Orkiestracja w Lakeflow Jobs	12
MODUŁ 06	Unity Catalog	14
MODUŁ 07	Dashboardy AI/BI i Genie Agents, opcjonalnie	16
ZAŁĄCZNIK A	Połączenie z ADLS Gen2	17
ZAŁĄCZNIK B	Ściągawka SQL	18
ZAŁĄCZNIK C	Dokumentacja	19

MODUŁ 01

Platforma i workspace

Architektura lakehouse

Lakehouse łączy otwarte i tanie przechowywanie danych, typowe dla data lake, z niezawodnością i nadzorem charakterystycznymi dla hurtowni danych. Databricks buduje go z trzech warstw.

Warstwa	Technologia	Zastosowanie
Przechowywanie	Delta Lake w chmurowym magazynie obiektów	Transakcje ACID, wersjonowanie, kontrola schematu
Obliczenia	Klastry Spark, SQL warehouse, serverless	Przetwarzanie wsadowe i strumieniowe, uczenie maszynowe, analityka SQL
Nadzór	Unity Catalog	Kontrola dostępu, lineage, wyszukiwanie danych

Zasoby obliczeniowe

Typ	Typowe zastosowanie	Działanie
Klaster all-purpose	Interaktywne notebooki i development	Działa do zatrzymania; opłata za czas działania
Klaster job	Zadania harmonogramowane i automatyczne	Tworzony na czas uruchomienia i usuwany po jego zakończeniu
SQL warehouse	Edytor SQL i narzędzia BI przez JDBC lub ODBC	Wstrzymuje się przy braku ruchu; zoptymalizowany pod równoległe zapytania
Serverless	Notebooki, zadania i SQL bez zarządzania klastrem	Startuje w kilka sekund; opłata za sekundy użycia

Przy nowych pracach serverless jest zwykle najlepszym wyborem domyślnym: startuje w kilka sekund, nie wymaga konfiguracji i nie generuje kosztów w czasie bezczynności.

Ścieżki do plików: wolumeny zamiast DBFS

	DBFS (legacy)	Wolumen Unity Catalog
Ścieżka	`dbfs:/…`, `dbfs:/mnt/…`	`/Volumes/<catalog>/<schema>/<volume>`
Nadzór	Brak; dostęp ma każdy użytkownik workspace	Uprawnienia Unity Catalog, audyt, lineage
Nowe workspace'y	Niedostępne	Domyślnie

> DBFS root i mounty są wycofywane. Konta Azure Databricks utworzone po 18 grudnia 2025 r. nie mają do nich dostępu, podobnie jak do Hive Metastore i klastrów no-isolation shared. Od 8 października 2026 r. każdy nowy workspace, w dowolnym koncie, powstaje bez tych funkcji i bez wersji Databricks Runtime starszych niż 13.3 LTS. Istniejące workspace'y je zachowują. W nowym kodzie używaj ścieżek wolumenów.

Polecenia magiczne

Polecenie	Działanie
`%sql`	Wykonuje komórkę jako SQL
`%python`	Wykonuje komórkę jako Python
`%sh`	Uruchamia polecenia powłoki na driverze
`%pip install <package>`	Instaluje bibliotekę na czas sesji notebooka
`%run ./path/notebook`	Uruchamia inny notebook w bieżącym kontekście
`%fs ls /Volumes/…`	Wyświetla pliki w wolumenie
`%skip`	Pomija komórkę przy uruchamianiu całego notebooka

Genie Code, asystent w notebookach

Genie Code, wcześniej Databricks Assistant, pisze, wyjaśnia i poprawia kod w notebookach oraz w SQL Editorze. Korzysta z metadanych Unity Catalog, więc jego podpowiedzi odnoszą się do tabel i kolumn w Twoim katalogu.

Czynność	Sposób
Polecenie w komórce	`Cmd+I` w macOS, `Ctrl+I` w Windows
Wyjaśnienie kodu	`/explain`
Naprawa błędu	`/fix` albo **Diagnose error** przy komórce z błędem
Dokumentacja kodu	`/doc`
Szybsza wersja kodu	`/optimize`
Formatowanie	`/prettify`

Genie Code wykonuje też zadania wieloetapowe, na przykład eksplorację tabeli i napisanie kodu, i prosi o zgodę, zanim cokolwiek uruchomi. Należy do rodziny Genie razem z Genie Agents (pytania w języku naturalnym o wybrane tabele, M07) i Genie One (czat dla użytkowników biznesowych).

> Przed uruchomieniem wygenerowanego kodu zawsze go przeczytaj, zwłaszcza jeśli zapisuje lub usuwa dane.

Widgety (BONUS)

```
dbutils.widgets.text("env", "dev", "Environment")
dbutils.widgets.dropdown("format", "delta", ["delta", "parquet", "csv"])
env = dbutils.widgets.get("env")
dbutils.widgets.removeAll()
```

MODUŁ 02

Architektura Spark

Driver i executory

Element	Rola
Driver	Wykonuje Twój kod, buduje plan wykonania, planuje zadania i zbiera wyniki. Jeden na klaster.
Executor	Wykonuje zadania równolegle, każde na jednej partycji w pamięci lub na dysku. Kilka na klaster.
Partycja	Fragment danych przetwarzany przez jedno zadanie. Domyślny rozmiar docelowy to 128 MB.

Transformacje i akcje

Pojęcie	Znaczenie	Przykłady
Transformacja	Leniwa: definiuje nowy DataFrame, nie obliczając go	`filter`, `select`, `join`, `groupBy`
Akcja	Uruchamia obliczenia i zwraca lub zapisuje wynik	`count()`, `show()`, `write`, `collect()`
Transformacja wąska	Działa w obrębie partycji; dane nie są przenoszone	`filter`, `select`
Transformacja szeroka	Rozdziela dane między partycje (shuffle)	`groupBy`, `join`, `distinct`

DAG, etapy i zadania (BONUS)

Spark zamienia ciąg transformacji w skierowany graf acykliczny (DAG). Graf jest dzielony na etapy (stages) przy każdym shuffle, a każdy etap wykonuje się jako jedno zadanie na partycję. Shuffle przesyła dane przez sieć między executorami i w większości zadań jest najkosztowniejszą operacją.

Catalyst i adaptive query execution (BONUS)

Mechanizm	Kiedy działa	Co robi
Optymalizator Catalyst	Podczas planowania	Przekształca plan logiczny w fizyczny: przenosi filtry bliżej źródła, pomija zbędne kolumny, upraszcza wyrażenia stałe
Adaptive query execution (AQE)	Podczas wykonania	Zmienia plan na podstawie rzeczywistych statystyk shuffle: łączy małe partycje, zmienia strategię joinów, dzieli partycje z przekosem danych

AQE jest domyślnie włączone w Databricks Runtime. DataFrame API i `spark.sql()` kompilują się do tego samego planu, więc wybierz zapis, który jest czytelniejszy.

MODUŁ 03

ELT i ingestia

Odczyt plików

```
# CSV: an explicit schema avoids a second pass over large files
df = (spark.read.format("csv")
      .option("header", True)
      .option("sep", ",")
      .option("nullValue", "NULL")
      .schema(my_schema)
      .load("/Volumes/<catalog>/<schema>/<volume>/customers.csv"))

# JSON Lines (one record per line, the default; the course data uses it)
df = spark.read.json("/Volumes/…/orders/")
# Only for a single JSON document spread over several lines
df = spark.read.option("multiLine", True).json("/Volumes/…/orders.json")

# Parquet and Delta carry their own schema
df = spark.read.parquet("/Volumes/…/products/")
df = spark.read.table("catalog.schema.table")
```

```
-- Ten sam odczyt JSON w SQL (read_files)
SELECT * FROM read_files('/Volumes/<catalog>/<schema>/<volume>/orders/', format => 'json');
```

Eksploracja DataFrame

Metoda	Wynik
`display(df)`	Interaktywna tabela; wykresy i profil danych są dostępne w menu +
`df.show(5, truncate=False)`	Podgląd w formie tekstu
`df.printSchema()`	Nazwy i typy kolumn
`df.describe()`	Liczność, średnia, odchylenie standardowe, minimum i maksimum
`df.summary()`	To samo co `describe()`, a do tego percentyle 25, 50 i 75

Najczęstsze transformacje

Operacja	Składnia
Wybór kolumn	`df.select("col1", "col2")`
Dodanie lub zmiana kolumny	`df.withColumn("new", col("x") * 2)`
Zmiana nazwy kolumny	`df.withColumnRenamed("old", "new")`
Filtrowanie wierszy	`df.filter(col("age") > 18)`
Wartość warunkowa	`when(col("x") > 0, "pos").otherwise("neg")`
Usunięcie kolumny	`df.drop("col")`
Zmiana typu	`col("id").cast("integer")`
Bezpieczna zmiana typu	`expr("try_cast(x AS INT)")`; SQL `try_cast(x AS INT)` zwraca NULL zamiast błędu CAST_INVALID_INPUT
Zapis do tabeli Delta	`df.write.mode("overwrite").saveAsTable("catalog.bronze.orders")`; SQL `CREATE TABLE ... AS SELECT`
Usunięcie duplikatów	`df.dropDuplicates(["col1", "col2"])`
Sortowanie	`df.orderBy(col("date").desc())`
Obsługa wartości null	`df.fillna({"col": 0})`, `df.dropna()`

Agregacja

```
from pyspark.sql.functions import count, sum, avg, col

(df.groupBy("region")
   .agg(count("*").alias("orders"),
        sum("amount").alias("revenue"),
        avg("amount").alias("avg_order"))
   .filter(col("orders") > 100))        # equivalent of SQL HAVING
```

Joiny (BONUS)

Typ	Zwracane wiersze
`inner`	Wiersze pasujące w obu DataFrame'ach
`left`	Wszystkie wiersze z lewej strony, z dopasowaniami z prawej lub wartościami null
`right`	Wszystkie wiersze z prawej strony, z dopasowaniami z lewej lub wartościami null
`full`	Wszystkie wiersze z obu stron
`cross`	Wszystkie kombinacje wierszy

```
orders.join(customers, orders.customer_id == customers.id, "left")
```

Widoki tymczasowe

```
df.createOrReplaceTempView("orders_view")
spark.sql("SELECT region, SUM(amount) FROM orders_view GROUP BY region").show()
```

Zagnieżdżony JSON (BONUS)

```
from pyspark.sql.functions import explode, get_json_object, col

df.withColumn("item", explode(col("items")))                              # array to rows
df.withColumn("city", get_json_object(col("json_str"), "$.address.city"))
```

MODUŁ 04

Delta Lake

Delta Lake przechowuje dane w plikach Parquet wraz z dziennikiem transakcji w katalogu `_delta_log/`. Każda zmiana trafia do dziennika i to właśnie umożliwia transakcje ACID, time travel oraz kontrolę schematu.

Gwarancje ACID

Własność	Znaczenie
Atomowość (atomicity)	Zapis kończy się w całości albo nie zostawia żadnego śladu
Spójność (consistency)	Schemat i ograniczenia są sprawdzane przy każdym zapisie
Izolacja (isolation)	Równolegle czytający i zapisujący nie widzą niedokończonych zmian innych
Trwałość (durability)	Zatwierdzone dane przetrwają awarię

Modyfikacja danych

```
INSERT INTO catalog.schema.orders VALUES (1001, 'C001', 99.99, current_date());

UPDATE catalog.schema.orders SET status = 'shipped' WHERE order_id = 1001;

DELETE FROM catalog.schema.orders WHERE status = 'cancelled';

-- Upsert: update matches, insert new rows, delete rows missing from the source
MERGE INTO target t
USING source s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.val = s.val
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

Time travel

```
DESCRIBE HISTORY my_table;

SELECT * FROM my_table VERSION AS OF 5;
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';

RESTORE TABLE my_table TO VERSION AS OF 5;
```

```
df = spark.read.option("versionAsOf", 5).table("my_table")
```

VACUUM

```
VACUUM my_table DRY RUN;             -- list the files that would be removed
VACUUM my_table;                     -- remove files older than 7 days (the default)
VACUUM my_table RETAIN 168 HOURS;    -- the same, with the retention stated explicitly
```

> VACUUM usuwa stare pliki danych, a nie `_delta_log`. Usuniętych plików nie da się odzyskać, a wersje tabeli, które od nich zależały, przestają być dostępne przez time travel i `RESTORE`. Na tabelach czytanych równolegle utrzymuj retencję co najmniej 7 dni.

Liquid clustering i predictive optimization

```
CREATE TABLE my_table (order_id STRING, order_date DATE, amount DOUBLE)
CLUSTER BY (order_date);                    -- cluster by columns you filter on
ALTER TABLE my_table CLUSTER BY AUTO;       -- let Databricks choose the keys
OPTIMIZE my_table;                          -- compact and cluster new data
```

Liquid clustering to zalecany układ danych dla nowych tabel; zastępuje partycjonowanie i Z-ordering. Przy włączonym predictive optimization Databricks sam uruchamia na tabelach zarządzanych Unity Catalog utrzymanie, takie jak `OPTIMIZE` i `VACUUM`. `OPTIMIZE ... ZORDER BY (col)` zostaje dla starszych tabel bez clusteringu.

Tabele zarządzane i zewnętrzne

	Tabela zarządzana	Tabela zewnętrzna
Lokalizacja danych	Magazyn zarządzany przez Unity Catalog	Wskazana ścieżka w chmurze (`abfss://`, `s3://`, `gs://`)
`DROP TABLE`	Usuwa metadane i pliki danych	Usuwa tylko metadane; pliki pozostają
Kiedy stosować	Domyślnie	Gdy dane muszą pozostać w istniejącej lokalizacji

MODUŁ 05

Orkiestracja w Lakeflow Jobs

Pojęcia

Termin	Znaczenie
Job	Jednostka uruchamiana według harmonogramu lub na żądanie, złożona z jednego lub kilku zadań
Task	Pojedynczy krok: notebook, skrypt Python, zapytanie SQL, pipeline i inne
Zależność	Kolejność wykonywania zadań; zadania joba tworzą DAG
Repair run	Ponowne uruchomienie tylko zadań zakończonych błędem i zadań od nich zależnych
Klaster job	Zasoby tworzone na czas uruchomienia i usuwane po jego zakończeniu

Wyzwalacze

Wyzwalacz	Uruchamia job
Scheduled	Według harmonogramu cron
File arrival	Gdy we wskazanej lokalizacji pojawią się nowe pliki
Continuous	Ponownie, zaraz po zakończeniu poprzedniego uruchomienia
Ręcznie	Na żądanie, z interfejsu lub przez API

Wyrażenia cron

Databricks używa formatu Quartz, który zaczyna się od pola sekund.

```
sek.  min.  godz.  dzień mies.  mies.  dzień tyg.
 0     30     2         *         *         ?        codziennie o 02:30
 0      0     8         ?         *      MON-FRI     w dni robocze o 08:00
 0      0     0         1         *         ?        pierwszego dnia miesiąca o północy
```

Przekazywanie wartości między zadaniami

```
# Read a job or task parameter (the default is used outside a job)
dbutils.widgets.text("source_table", "samples.nyctaxi.trips")
source_table = dbutils.widgets.get("source_table")

# Task "validate": publish a value for downstream tasks
dbutils.jobs.taskValues.set(key="row_count", value=row_count)

# Task "transform": read it; debugValue is used when the notebook runs outside a job
row_count = dbutils.jobs.taskValues.get(taskKey="validate", key="row_count", debugValue=-1)
```

Parametry joba trafiają do każdego zadania. Jeśli parametr joba i parametr zadania mają ten sam klucz, wygrywa parametr joba.

Lakeflow Spark Declarative Pipelines (BONUS)

Lakeflow Spark Declarative Pipelines, dawniej Delta Live Tables, pozwala deklaratywnie definiować tabele; zależności i kolejność odświeżania ustala Databricks. Pipeline tworzymy w **Jobs & Pipelines**, potem **Create**, **ETL pipeline**.

Obiekt	Działanie
Streaming table	Pobiera dane przyrostowo, przetwarzając tylko dane nowe od ostatniej aktualizacji
Materialized view	Przechowuje wynik zapytania i odświeża go przy każdej aktualizacji pipeline'u
Widok	Nazwane zapytanie używane wewnątrz pipeline'u; nie jest zapisywane

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT * FROM STREAM read_files('/Volumes/<catalog>/<schema>/<volume>/orders/', format =>
'json');

CREATE OR REFRESH MATERIALIZED VIEW silver_orders
AS SELECT order_id, customer_id, total_amount FROM bronze_orders WHERE total_amount > 0;
```

Tabele z tego samego pipeline'u wskazujemy po nazwie; dawny prefiks `LIVE.` nie jest potrzebny.

Monitorowanie nieudanych uruchomień (demo trenera)

```
SELECT *
FROM system.lakeflow.job_run_timeline
WHERE result_state = 'FAILED'
  AND period_start_time >= current_timestamp() - INTERVAL 7 DAYS;
```

Odczyt tabel systemowych wymaga dostępu do katalogu `system`.

MODUŁ 06

Unity Catalog

Hierarchia obiektów

```
Metastore           jeden na region, wspólny dla workspace'ów w tym regionie
└── Catalog         na przykład dev, prod, sandbox
    └── Schema      logiczna grupa obiektów, odpowiednik bazy danych
        └── Tabela, widok, wolumen, funkcja
```

Odwołuj się do obiektów pełną, trzyczłonową nazwą `catalog.schema.table`, na przykład `main.retail.orders`.

Nadawanie uprawnień

Uprawnienia trzeba nadać na każdym poziomie, zaczynając od katalogu.

```
GRANT USE CATALOG ON CATALOG main              TO `retailhub_analysts`;
GRANT USE SCHEMA  ON SCHEMA  main.retail       TO `retailhub_analysts`;
GRANT SELECT      ON TABLE   main.retail.orders TO `retailhub_analysts`;

SHOW GRANTS ON TABLE main.retail.orders;

REVOKE SELECT ON TABLE main.retail.orders FROM `retailhub_analysts`;
```

Uprawnienie	Nadawane na	Pozwala na
`USE CATALOG`	Katalog	Otwarcie katalogu; wymagane jako pierwsze
`USE SCHEMA`	Schemat	Otwarcie schematu
`SELECT`	Tabela, widok lub schemat (dla wszystkich jego tabel)	Odczyt danych
`MODIFY`	Tabela	Wstawianie, aktualizację i usuwanie danych
`CREATE TABLE`	Schemat	Tworzenie tabel
`MANAGE`	Dowolny obiekt	Nadawanie i odbieranie uprawnień do tego obiektu
`ALL PRIVILEGES`	Dowolny obiekt	Większość uprawnień, ale nie `MANAGE`

Uprawnienia do obiektu może nadawać tylko jego właściciel, administrator metastore albo posiadacz `MANAGE`. Samo `ALL PRIVILEGES` nie wystarcza.

Grupy

Nadawaj uprawnienia grupom, a nie pojedynczym użytkownikom. Unity Catalog akceptuje wyłącznie grupy kontowe.

	Grupa kontowa	Grupa lokalna workspace
Tworzona w	Konsoli konta albo w ustawieniach workspace, w sekcji Identity and access	Workspace Groups API albo poleceniem SQL `CREATE GROUP`
Może otrzymać uprawnienia Unity Catalog	Tak	Nie; nadanie kończy się błędem `PRINCIPAL_DOES_NOT_EXIST`

> Polecenie SQL `CREATE GROUP` wykonuje się bez błędu, ale tworzy grupę lokalną workspace. Grupy twórz w konsoli konta albo w ustawieniach workspace.

Zewnętrzne magazyny danych (BONUS)

Obiekt	Zastosowanie
Storage credential	Przechowuje tożsamość w chmurze, na przykład Azure Access Connector, używaną do dostępu do magazynu
External location	Łączy ścieżkę w magazynie z credentialem i kontroluje `READ FILES` oraz `WRITE FILES`
Tabela zewnętrzna	Tabela, której pliki są przechowywane w external location

MODUŁ 07

Dashboardy AI/BI i Genie Agents, opcjonalnie

Dashboardy AI/BI

Dashboardy prezentują dane z Unity Catalog jako wykresy, liczniki i tabele, z wyborem dat, filtrami rozwijanymi i filtrowaniem krzyżowym między widżetami. Opublikowany dashboard ma link do udostępniania, nie wymaga od odbiorców uruchamiania zapytań, może być wysyłany e-mailem według harmonogramu i osadzany w innych aplikacjach.

Genie Agents

Genie Agent (wcześniej Genie space) odpowiada na pytania zadawane językiem naturalnym, generując i wykonując SQL na skonfigurowanych dla niego tabelach Unity Catalog. Tworzymy go z pozycji **Genie Agents** w menu bocznym; potrzebny jest SQL warehouse typu pro lub serverless oraz włączone w workspace partner-powered AI features. Pamięta kontekst rozmowy, więc działają pytania uzupełniające, a odpowiedzi można zapisać jako widżety dashboardu. Genie dobrze sprawdza się przy eksploracyjnych, samoobsługowych pytaniach; gorzej radzi sobie z wieloetapową logiką i raportami o ściśle określonym formacie.

Wybór narzędzia

Potrzeba	Narzędzie
Zatwierdzony dashboard KPI odświeżany codziennie	Dashboard AI/BI
Doraźne pytanie, np. o dziesięć najlepiej sprzedających się produktów w zeszłym miesiącu	Genie Agent
Eksploracja danych przed zbudowaniem dashboardu	SQL Editor, który ma własne wizualizacje

ZAŁĄCZNIK A

Połączenie z ADLS Gen2

Krok	Gdzie	Czynność
1	Azure Portal, zarządzana grupa zasobów Databricks	Skopiuj identyfikator zasobu Access Connectora (`Microsoft.Databricks/accessConnectors/<name>`)
2	Konto magazynu, Access control (IAM)	Przypisz rolę **Storage Blob Data Contributor** tożsamości zarządzanej Access Connectora
3	Catalog Explorer, External data, Credentials	Utwórz storage credential z identyfikatorem zasobu Access Connectora
4	Catalog Explorer, External locations	Utwórz external location z adresu `abfss://` i credentialu
5	External location, Permissions	Nadaj `READ FILES`, `WRITE FILES` lub `CREATE EXTERNAL TABLE` zgodnie z potrzebami

```
SHOW EXTERNAL LOCATIONS;
LIST 'abfss://container@account.dfs.core.windows.net/';
```

> Dla produkcyjnych źródeł tylko do odczytu przypisz rolę Storage Blob Data Reader zamiast Contributor i nadawaj pojedyncze uprawnienia Unity Catalog zamiast `ALL PRIVILEGES`.

ZAŁĄCZNIK B

Ściągawka SQL

Delta Lake

```
DESCRIBE DETAIL my_table;
DESCRIBE HISTORY my_table;
ALTER TABLE my_table CLUSTER BY (col);
OPTIMIZE my_table;
VACUUM my_table RETAIN 168 HOURS;
RESTORE TABLE my_table TO VERSION AS OF 3;
SELECT * FROM my_table VERSION AS OF 2;
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';
```

Unity Catalog

```
SHOW CATALOGS;
SHOW SCHEMAS IN my_catalog;
SHOW TABLES IN my_catalog.my_schema;
DESCRIBE TABLE EXTENDED my_catalog.my_schema.my_table;
SHOW GRANTS ON TABLE my_catalog.my_schema.my_table;
```

ZAŁĄCZNIK C

Dokumentacja

Temat	Adres
Dokumentacja Databricks	docs.databricks.com
Delta Lake	docs.delta.io
Apache Spark	spark.apache.org/docs/latest
Lakeflow Jobs	docs.databricks.com/en/workflows/index.html
Lakeflow Spark Declarative Pipelines	learn.microsoft.com/azure/databricks/ldp/
Unity Catalog	docs.databricks.com/en/data-governance/unity-catalog/index.html
Uprawnienia w Unity Catalog	docs.databricks.com/en/data-governance/unity-catalog/manage-privileges/index.html
Storage credentials i external locations	docs.databricks.com/en/connect/unity-catalog/external-locations.html
Serverless	docs.databricks.com/en/compute/serverless.html
Adaptive query execution	docs.databricks.com/en/optimizations/aqe.html
Liquid clustering	learn.microsoft.com/azure/databricks/tables/clustering
Predictive optimization	learn.microsoft.com/azure/databricks/optimizations/predictive-optimization
Dashboardy AI/BI	docs.databricks.com/en/dashboards/index.html
Genie Agents	learn.microsoft.com/azure/databricks/genie-agents/
Genie Code	docs.databricks.com/en/genie-code/
Wycofanie DBFS i innych funkcji legacy	docs.databricks.com/en/admin/account-settings/legacy-features.html